

Inventory Management System

Design Document & Architecture Plan — Week 1

Junior Java Developer Internship (YuvaIntern — NSDC Job Simulation)

Prepared by: Pulkit | Author's project GitHub: pulkitcode-bit

1. Introduction and Objective

This document presents the conceptual architecture and design plan for a Java-based Inventory Management System (IMS) intended for a small retail business. The goal of Week 1 is not to write production code, but to establish a sound architectural foundation — the module boundaries, the design patterns that will govern how those modules interact, and the data model they will operate on — so that later weeks (core implementation, testing, refactoring, and deployment) build on a deliberate plan rather than ad-hoc code.

A small retail business typically needs to track what products it sells, how much of each is in stock, how orders (sales or purchase) move that stock, and how the owner can see summarized reports (best-sellers, low-stock items, stock valuation) to make purchasing decisions. The system therefore needs to be correct on inventory counts above all else — a wrong stock count is worse than a slow report — which shapes several of the design decisions below, particularly around where stock updates are allowed to happen.

2. Requirement Analysis

2.1 Functional Requirements

- **Product Management:** create, update, deactivate, and search products; each product belongs to a category and has a unit price and a reorder level.
- **Stock Control:** view current stock per product/location, record stock-in (purchase/receipt) and stock-out (sale/damage) movements, and prevent stock from going negative.
- **Order Processing:** create a sales order with one or more line items, validate stock availability at order time, and automatically deduct stock when an order is confirmed.
- **Reporting:** generate a low-stock report (items at or below reorder level), a sales summary for a date range, and a stock valuation report (quantity $\times$ unit price).
- **User Management & Access Control:** authenticate users and restrict stock adjustments and product edits to STAFF/ADMIN roles.
- **Notifications:** alert relevant staff (via log entry, and later email) when a product's stock crosses its reorder level.

2.2 Non-Functional Requirements

- **Maintainability:** modules should be loosely coupled so a change in, say, reporting does not require touching order processing.
- **Data consistency:** stock deduction and order creation must be atomic (a transaction) to avoid partial updates.
- **Extensibility:** it should be possible to add a new report type or a new discount rule without modifying existing classes, only by adding new ones.
- **Testability:** business rules should live in plain Java service classes that can be unit-tested without a running web server or database.
- **Reasonable performance for the target scale:** a single small-to-medium retail store, not a multi-warehouse enterprise deployment.

3. High-Level Architecture

The system follows a classic layered (MVC-influenced) architecture on top of Spring Boot, which matches how the rest of this internship's stack (Java, Spring Boot, MySQL) is expected to be used. Requests flow strictly downward — Controller → Service → Repository → Database — and responses flow back up the same path. No layer is allowed to skip another (a Controller never talks to a Repository directly), which keeps responsibilities clean and makes each layer replaceable on its own.

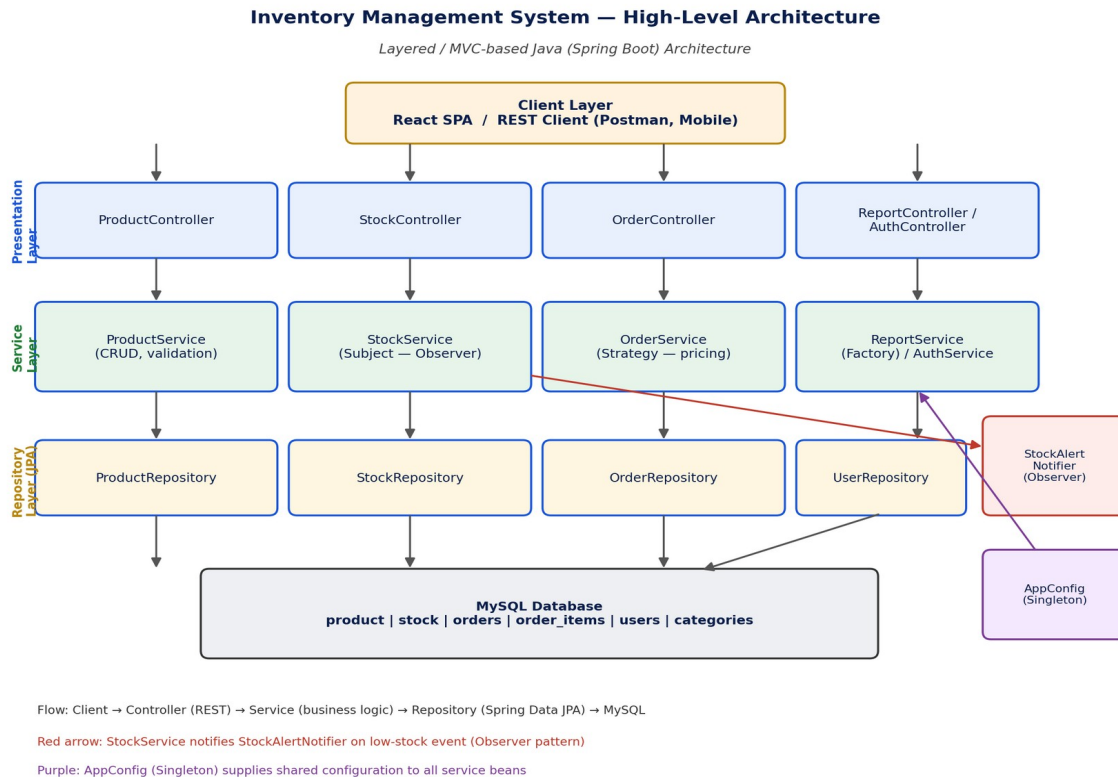


Figure 1: Layered architecture of the Inventory Management System, showing the four modules (Product, Stock, Order, Report/Auth) across the Presentation, Service, and Repository layers, plus the cross-cutting Observer and Singleton components.

3.1 Layer Responsibilities

- Presentation Layer (Controllers): exposes REST endpoints (e.g. GET /api/products, POST /api/orders), performs request validation, and maps DTOs to/from JSON. Contains no business logic.
- Service Layer: owns all business rules — stock-availability checks, reorder-level comparisons, discount/tax calculation, and orchestrating multi-step operations inside a single @Transactional boundary.
- Repository Layer: Spring Data JPA interfaces that talk to MySQL; each repository is scoped to one aggregate root (Product, Stock, Order, User).
- Cross-cutting components: StockAlertNotifier (an Observer of StockService) and AppConfig (a Singleton holding shared settings such as the default reorder threshold and tax rate) sit beside the main flow and are consumed by the service layer rather than being part of the request/response chain.

4. Module Descriptions

4.1 Product Management Module

Owns the product catalogue: SKU, name, description, category, unit price, and reorder level. ProductService enforces uniqueness of SKU and prevents deletion of a product that has existing stock or open orders (it is deactivated instead, so historical orders still resolve correctly).

4.2 Stock Control Module

Owns the quantity_on_hand for each product (optionally per warehouse location in a future iteration). Every stock change goes through StockService.adjustStock(), never directly through the repository, so that the reorder-level check and the Observer notification always fire consistently regardless of who calls it.

4.3 Order Processing Module

Owns order headers and line items. OrderService validates that requested quantities are available (delegating the check to StockService), computes totals using a pluggable PricingStrategy, and — on confirmation — calls StockService to deduct stock and persists the order, all inside one transaction so a failure partway through rolls back cleanly.

4.4 Reporting Module

Read-only by design: ReportService only queries existing data (via the repositories) and never mutates it. A ReportFactory selects the concrete report-builder class based on a requested report type, keeping ReportController and ReportService free of long if/else chains as new report types are added.

4.5 User Management & Auth Module

A minimal AuthService handles login and issues a session/JWT token; role (ADMIN vs STAFF) is checked at the Controller/Service boundary using Spring Security annotations, so unauthorized stock edits are rejected before reaching business logic.

5. Design Patterns and Rationale

The table below summarizes the design patterns chosen for this system and why each one fits a specific problem in the architecture, rather than being used for its own sake.

Design Pattern	Applied In	Rationale
Layered Architecture (MVC-style)	Whole application (Controller → Service → Repository)	Separates HTTP handling, business rules, and persistence so each layer can change independently and be unit-tested in isolation.
Repository Pattern	ProductRepository, StockRepository, OrderRepository, UserRepository	Abstracts data-access logic behind an interface (Spring Data JPA), so the service layer never depends on SQL or JPA specifics directly.
DTO (Data Transfer Object)	ProductDTO, OrderRequestDTO, StockAlertDTO	Prevents leaking JPA entities to the client; lets the API shape differ from the DB schema and avoids lazy-loading serialization issues.
Singleton	AppConfig, DataSource bean	Spring beans are singletons by default; used deliberately for shared, stateless configuration (currency, low-stock threshold, tax rate).
Observer	StockService (subject) →	When stock for a product drops below

	StockAlertNotifier (observer)	a threshold, StockService publishes an event; one or more listeners (email/log notifier) react without StockService knowing who is listening.
Factory	ReportFactory inside ReportService	Report type (Sales / Low-Stock / Valuation) is chosen at runtime; the factory hides which concrete Report class gets built.
Strategy	PricingStrategy inside OrderService	Discount / tax calculation rules differ by order type or customer tier; the strategy interface lets the algorithm be swapped without changing OrderService.

6. Technology Stack

- Language & Framework: Java 17, Spring Boot (Spring Web, Spring Data JPA, Spring Security)
- Database: MySQL 8
- Build tool: Maven
- Testing: JUnit 5 and Mockito (for Week 3's unit-testing task)
- Optional client: React SPA consuming the REST API (kept decoupled from the backend)

7. Database Schema Overview

A brief relational schema is outlined below; this will be refined into full DDL once implementation begins in Week 2.

Entity	Key Fields	Notes
product	id, sku, name, category_id, unit_price, reorder_level	Core catalogue item; reorder_level feeds the Observer alert.
category	id, name	Simple lookup table for grouping products.
stock	id, product_id (FK), quantity_on_hand, warehouse_location, last_updated	One row per product per location; updated on every stock movement.
orders	id, order_date, customer_id, status, total_amount	Header row for a sales/purchase order.
order_items	id, order_id (FK), product_id (FK), quantity, unit_price	Line items; quantity here drives stock deduction.
users	id, username, password_hash, role	Drives AuthService / role-based access (ADMIN, STAFF).

8. Code Scaffolding / Pseudocode

The snippets below are structural stubs, not full implementations — they exist to lock in the module boundaries and method signatures decided above before any business logic is written.

```
// --- Repository layer ---
public interface ProductRepository extends JpaRepository<Product, Long> {
    Optional<Product> findBySku(String sku);
}

public interface StockRepository extends JpaRepository<Stock, Long> {
```

```

        Optional<Stock> findByProductId(Long productId);
    }

    // --- Service layer: Stock control with Observer notification ---
    public class StockService {
        private final StockRepository stockRepository;
        private final List<StockAlertListener> listeners; // Observer pattern

        @Transactional
        public void adjustStock(Long productId, int delta) {
            Stock stock = stockRepository.findByProductId(productId)
                .orElseThrow(() -> new ProductNotFoundException(productId));
            int newQty = stock.getQuantity() + delta;
            if (newQty < 0) throw new InsufficientStockException(productId);
            stock.setQuantity(newQty);
            stockRepository.save(stock);

            if (newQty <= stock.getProduct().getReorderLevel()) {
                listeners.forEach(l -> l.onLowStock(stock)); // notify observers
            }
        }
    }

    // --- Service layer: Order processing with Strategy pricing ---
    public class OrderService {
        private final StockService stockService;
        private final PricingStrategy pricingStrategy; // Strategy pattern

        @Transactional
        public Order placeOrder(OrderRequestDTO request) {
            for (OrderItemDTO item : request.getItems()) {
                stockService.adjustStock(item.getProductId(), -item.getQuantity());
            }
            BigDecimal total = pricingStrategy.calculateTotal(request);
            Order order = new Order(request, total);
            return orderRepository.save(order);
        }
    }

    // --- Service layer: Reporting via Factory ---
    public class ReportService {
        private final ReportFactory reportFactory; // Factory pattern

        public Report generate(ReportType type, ReportCriteria criteria) {
            ReportBuilder builder = reportFactory.getBuilder(type);
            return builder.build(criteria);
        }
    }

    // --- Cross-cutting: Singleton configuration ---
    @Component
    public class AppConfig {
        // Spring manages this as a singleton bean
        private final int defaultReorderThreshold = 10;
        private final BigDecimal taxRate = new BigDecimal("0.18");
        // getters only – immutable shared configuration
    }

```

9. Conclusion

This design lays out four cleanly separated modules — Product, Stock, Order, and Reporting — communicating through a strict Controller-Service-Repository flow, with an Observer used for low-stock notification, a Strategy for pricing, a Factory for report generation, and a Singleton for shared configuration. Every stock mutation is funneled through one method (`StockService.adjustStock`), which is the single place that enforces the non-negative-stock rule and triggers alerts — this is deliberately the most protected part of the design, since incorrect stock counts are the most damaging failure mode for a real retail business. Week 2 will implement this plan feature-by-feature, starting with the Product and Stock modules, so that Order processing (which depends on both) has a working foundation to build on.